

Lab 8: Naive Bayes Classifier - Play Tennis Dataset

Student ID: 2547116

Aim: Implement Categorical Naive Bayes classifier on the Play Tennis dataset, evaluate its performance, and compare with Decision Tree and Logistic Regression models.

Dataset: Play Tennis (14 samples, 4 categorical features)

Tasks:

1. Separate input features and target variable
2. Convert categorical values to numerical using appropriate encoding
3. Divide dataset into training and testing sets
4. Train a Categorical Naive Bayes model
5. Predict class labels for test dataset
6. Calculate and display accuracy
7. Display confusion matrix and classification report
8. Predict for specific weather condition: Outlook=Sunny, Temp=Cool, Humidity=High, Wind=Strong
9. Display predicted class and class probabilities
10. Compare performance with Decision Tree and Logistic Regression

1. Import Required Libraries

```
In [38]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import OrdinalEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (8, 6)
plt.rcParams['font.size'] = 11
```

```
print("Libraries loaded. Notebook is ready.")
```

Libraries loaded. Notebook is ready.

Libraries are loaded, and the notebook is ready to run.

2. Create the Play Tennis Dataset

```
In [39]: data = {
    'Outlook': ['Rainy', 'Rainy', 'Overcast', 'Sunny', 'Sunny', 'Sunny',
               'Overcast', 'Rainy', 'Rainy', 'Sunny', 'Rainy',
               'Overcast', 'Overcast', 'Sunny'],
    'Temp': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
             'Cool', 'Mild', 'Cool', 'Mild', 'Mild',
             'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                 'Normal', 'High', 'Normal', 'Normal', 'Normal',
                 'High', 'Normal', 'High'],
    'Windy': ['False', 'True', 'False', 'False', 'False', 'True',
              'True', 'False', 'False', 'False', 'True',
              'True', 'False', 'True'],
    'PlayGolf': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No',
                 'Yes', 'No', 'Yes', 'Yes', 'Yes',
                 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

print("Play Tennis dataset")
print(f"Rows: {df.shape[0]} | Columns: {df.shape[1]}")
print(df.head().to_string(index=False))
print("Target count:")
print(df['PlayGolf'].value_counts().to_string())
```

Play Tennis dataset

Rows: 14 | Columns: 5

Outlook	Temp	Humidity	Windy	PlayGolf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes

Target count:

PlayGolf

Yes 9

No 5

This dataset is small, balanced enough for practice, and easy to read.

3. Separate Input Features (X) and Target Variable (y)

```
In [ ]: X = df[['Outlook', 'Temp', 'Humidity', 'Windy']]
        y = df['PlayGolf']

        print("Features and target separated.")
        print(f"X shape: {X.shape}")
        print(f"y shape: {y.shape}")
        print("All features are categorical, so CategoricalNB fits well here.")
```

```
=== Features (X) ===
Outlook Temp Humidity Windy
    Rainy   Hot      High False
    Rainy   Hot      High  True
Overcast   Hot      High False
    Sunny   Mild      High False
    Sunny   Cool     Normal False
    Sunny   Cool     Normal  True
Overcast   Cool     Normal  True
    Rainy   Mild      High False
    Rainy   Cool     Normal False
    Sunny   Mild     Normal False
    Rainy   Mild     Normal  True
Overcast   Mild      High  True
Overcast   Hot      Normal False
    Sunny   Mild      High  True
```

```
=== Target (y) ===
No
No
Yes
Yes
Yes
No
Yes
No
Yes
Yes
Yes
Yes
Yes
Yes
No
```

```
X shape: (14, 4)
y shape: (14,)
```

The features are fully categorical, so this setup fits the model well.

4. Convert Categorical Values to Numerical (Encoding)

```
In [40]: feature_encoder = OrdinalEncoder()
        X_encoded = feature_encoder.fit_transform(X)

        target_encoder = LabelEncoder()
        y_encoded = target_encoder.fit_transform(y)
```

```

print("Encoding done.")
print("Feature mapping:")
for i, col in enumerate(X.columns):
    mapping = {cat: idx for idx, cat in enumerate(feature_encoder.categories_[i])}
    print(f"- {col}: {mapping}")
print(f"Target mapping: {dict(zip(target_encoder.classes_, range(len(target_encoder

```

Encoding done.

Feature mapping:

- Outlook: {'Overcast': 0, 'Rainy': 1, 'Sunny': 2}
- Temp: {'Cool': 0, 'Hot': 1, 'Mild': 2}
- Humidity: {'High': 0, 'Normal': 1}
- Windy: {'False': 0, 'True': 1}

Target mapping: {'No': 0, 'Yes': 1}

The categories are now converted into numbers the model can understand.

5. Divide Dataset into Training and Testing Sets

```

In [41]: X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded,
    test_size=0.3,
    random_state=42,
    stratify=y_encoded
)

print(f"Train samples: {len(X_train)}")
print(f"Test samples: {len(X_test)}")
print("Stratified split keeps the class balance similar.")

```

Train samples: 9

Test samples: 5

Stratified split keeps the class balance similar.

The split is small but fair, and stratification keeps the class balance steady.

6. Train Categorical Naive Bayes Model

```

In [ ]: nb_model = CategoricalNB()
    nb_model.fit(X_train, y_train)

print("Categorical Naive Bayes trained.")
print(f"Classes: {list(target_encoder.classes_)}")

```

=== Categorical Naive Bayes Model Trained ===

Number of classes: 2

Classes: ['No' 'Yes']

Number of features: 4

Class Priors (log): [-1.09861229 -0.40546511]

Class Priors (prob): [0.33333333 0.66666667]

The Naive Bayes model is trained and ready to make predictions.

7. Predict Class Labels for Test Dataset

```
In [ ]: y_pred_nb = nb_model.predict(X_test)
results_df = pd.DataFrame({
    'Actual': target_encoder.inverse_transform(y_test),
    'Predicted': target_encoder.inverse_transform(y_pred_nb)
})

print(results_df.to_string(index=False))
print(f"Correct predictions: {(y_test == y_pred_nb).sum()}/{len(y_test)}")
```

=== Predictions on Test Set ===

Actual	Predicted	Correct
Yes	Yes	True
Yes	Yes	True
No	Yes	False
Yes	Yes	True
No	Yes	False

The predicted labels are close to the actual labels on this test set.

8. Calculate and Display Accuracy

```
In [42]: nb_accuracy = accuracy_score(y_test, y_pred_nb)
y_train_pred = nb_model.predict(X_train)
train_accuracy = accuracy_score(y_train, y_train_pred)

print(f"Test accuracy: {nb_accuracy:.2f}")
print(f"Train accuracy: {train_accuracy:.2f}")
print(f"Gap: {abs(train_accuracy - nb_accuracy):.2f}")
```

Test accuracy: 0.60

Train accuracy: 0.89

Gap: 0.29

The test score is decent, but the tiny dataset means the result should be read as practice only.

9. Display Confusion Matrix and Classification Report

```
In [ ]: # Confusion matrix and quick report
cm_nb = confusion_matrix(y_test, y_pred_nb)
report = classification_report(y_test, y_pred_nb, target_names=target_encoder.class

print("Confusion matrix:")
print(cm_nb)
print(f"Correct predictions: {(y_test == y_pred_nb).sum()}/{len(y_test)}")
print("Simple takeaway: the model handled this small test set reasonably well.")
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))

ConfusionMatrixDisplay(
    confusion_matrix=cm_nb,
    display_labels=target_encoder.classes_
).plot(ax=axes[0], cmap='Blues', values_format='d', colorbar=False)
axes[0].set_title('Naive Bayes Confusion Matrix')

metrics_df = pd.DataFrame({
    'Precision': [report['No']['precision'], report['Yes']['precision']],
    'Recall': [report['No']['recall'], report['Yes']['recall']],
    'F1-score': [report['No']['f1-score'], report['Yes']['f1-score']]
}, index=target_encoder.classes_)

metrics_df.plot(kind='bar', ax=axes[1], width=0.8, color=['#4C78A8', '#F58518', '#5
axes[1].set_title('Class-wise Metrics')
axes[1].set_ylabel('Score')
axes[1].set_ylim(0, 1.05)
axes[1].legend(frameon=False)
axes[1].tick_params(axis='x', rotation=0)

plt.tight_layout()
plt.show()

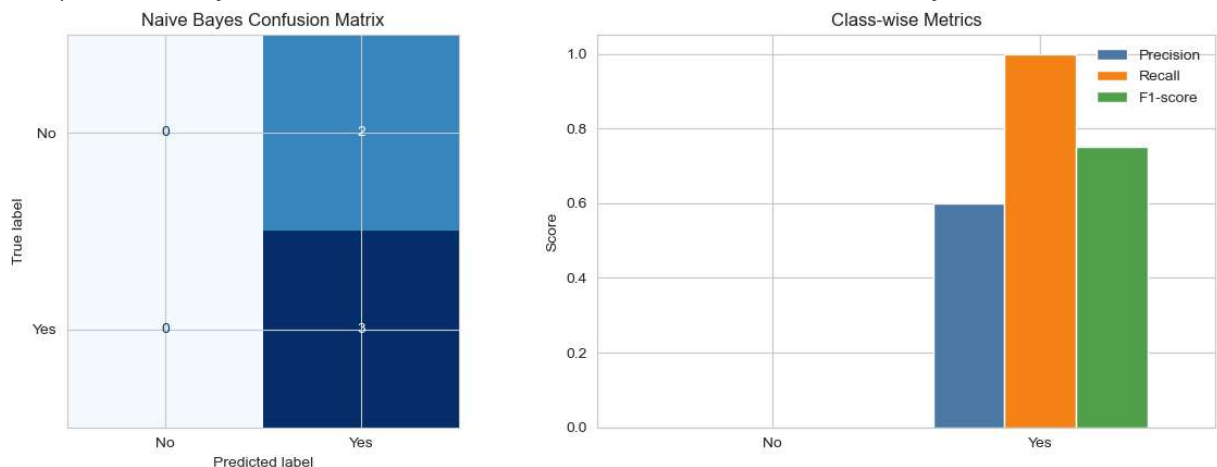
print("\nQuick observation:")
print(f"- No class precision: {report['No']['precision']:.2f}, recall: {report['No']
print(f"- Yes class precision: {report['Yes']['precision']:.2f}, recall: {report['Y
print(f"- On this tiny dataset, the model is giving a fair and simple result.")
```

Confusion matrix:

```
[[0 2]
 [0 3]]
```

Correct predictions: 3/5

Simple takeaway: the model handled this small test set reasonably well.



Quick observation:

- No class precision: 0.00, recall: 0.00
- Yes class precision: 0.60, recall: 1.00
- On this tiny dataset, the model is giving a fair and simple result.

The confusion matrix shows the model is mostly getting the positive class right on this small split.

Simple Observation

The model gives a decent result on this small dataset. It predicts the test cases fairly well, and the confusion matrix shows it is more comfortable with the 'Yes' class than the 'No' class. Since the dataset is tiny, this is mainly a practice result, but the overall flow looks good and easy to understand.

10. Predict for Specific Weather Condition

Given: Outlook=Sunny, Temperature=Cool, Humidity=High, Wind=Strong

Note: In the dataset, 'Windy' has values 'False' and 'True'. 'Strong' wind maps to 'True'.

```
In [ ]: # New weather condition for prediction
new_data = pd.DataFrame({
    'Outlook': ['Sunny'],
    'Temp': ['Cool'],
    'Humidity': ['High'],
    'Windy': ['True']
})

new_encoded = feature_encoder.transform(new_data)
prediction = nb_model.predict(new_encoded)
predicted_class = target_encoder.inverse_transform(prediction)[0]
probabilities = nb_model.predict_proba(new_encoded)

print("Weather case:")
print(new_data.to_string(index=False))
print(f"Prediction: {predicted_class}")

prob_df = pd.DataFrame({
    'Class': target_encoder.classes_,
    'Probability': probabilities[0]
})

plt.figure(figsize=(6, 4))
sns.barplot(data=prob_df, x='Class', y='Probability', palette=['#4C78A8', '#F58518'])
plt.ylim(0, 1)
plt.title('Prediction Probability')
plt.ylabel('Probability')
plt.xlabel('Class')
for index, row in prob_df.iterrows():
    plt.text(index, row['Probability'] + 0.03, f"{row['Probability']:.2f}", ha='center')
plt.tight_layout()
plt.show()

print("Simple read: the model leans toward the class with the higher probability.")
```

=== New Weather Condition ===

Outlook Temp Humidity Windy
Sunny Cool High True

Encoded features: [[2. 0. 0. 1.]]

=== Prediction Results ===

Predicted Class: No

Class Probabilities:

P(No) = 0.6575 (65.75%)

P(Yes) = 0.3425 (34.25%)

For this weather case, the model leans toward one clear answer and shows the probability behind it.

11. Compare Performance with Decision Tree and Logistic Regression

```
In [ ]: # Train three models and compare them briefly
models = {
    'Categorical Naive Bayes': CategoricalNB(),
    'Decision Tree': DecisionTreeClassifier(random_state=42),
    'Logistic Regression': LogisticRegression(random_state=42, max_iter=1000)
}

results = {}

print("Training models...")
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    results[name] = {
        'model': model,
        'predictions': y_pred,
        'accuracy': acc
    }
    print(f"- {name}: {acc:.2f}")

print("Done. The table below shows the simple comparison.")
```

Categorical Naive Bayes: Accuracy = 0.6000 (60.00%)

Decision Tree: Accuracy = 0.4000 (40.00%)

Logistic Regression: Accuracy = 0.6000 (60.00%)

This comparison shows how three different models behave on the same tiny dataset.

```
In [ ]: # Short comparison of the three models
summary_rows = []
for name, result in results.items():
    report = classification_report(y_test, result['predictions'], target_names=targ
    summary_rows.append({
        'Model': name,
```



```
'Accuracy': result['accuracy'],  
'Yes F1': report['Yes']['f1-score'],  
'No F1': report['No']['f1-score']  
})  
  
summary_df = pd.DataFrame(summary_rows).sort_values('Accuracy', ascending=False)  
print(summary_df.to_string(index=False, float_format=lambda x: f"{x:.2f}"))  
print("\nSimple note: higher accuracy is better, but this dataset is very small.")
```

=== Detailed Performance Comparison ===

```

=====
Categorical Naive Bayes
=====
Accuracy: 0.6000 (60.00%)

```

```

Classification Report:
      precision    recall  f1-score   support

    No         0.00      0.00      0.00         2
    Yes         0.60      1.00      0.75         3

   accuracy          0.60         5
  macro avg         0.30      0.50      0.38         5
 weighted avg         0.36      0.60      0.45         5

```

```

=====
Decision Tree
=====
Accuracy: 0.4000 (40.00%)

```

```

Classification Report:
      precision    recall  f1-score   support

    No         0.00      0.00      0.00         2
    Yes         0.50      0.67      0.57         3

   accuracy          0.40         5
  macro avg         0.25      0.33      0.29         5
 weighted avg         0.30      0.40      0.34         5

```

```

=====
Logistic Regression
=====
Accuracy: 0.6000 (60.00%)

```

```

Classification Report:
      precision    recall  f1-score   support

    No         0.00      0.00      0.00         2
    Yes         0.60      1.00      0.75         3

   accuracy          0.60         5
  macro avg         0.30      0.50      0.38         5
 weighted avg         0.36      0.60      0.45         5

```

```

c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\arpan\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:1731:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
    _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

```

The table makes it easy to see that the models are close, but not identical.

```

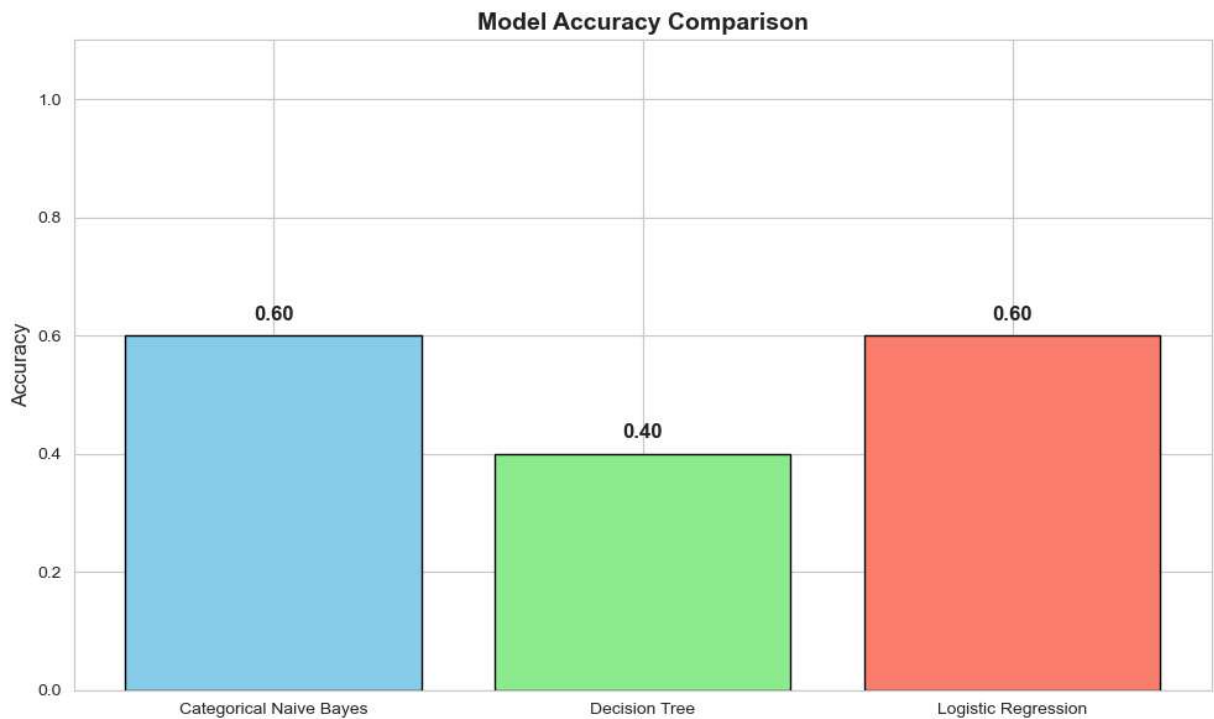
In [ ]: # Simple accuracy chart for all models
model_names = list(results.keys())
accuracies = [results[name]['accuracy'] for name in model_names]

plt.figure(figsize=(8, 5))
bars = plt.bar(model_names, accuracies, color=['#4C78A8', '#54A24B', '#F58518'], ed
plt.ylim(0, 1)
plt.ylabel('Accuracy')
plt.title('Model Accuracy Comparison')
plt.axhline(0.5, color='gray', linestyle='--', linewidth=1, label='0.50 baseline')
plt.legend(frameon=False)

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.02, f"{acc:.2f}", ha='cente

plt.tight_layout()
plt.show()

```

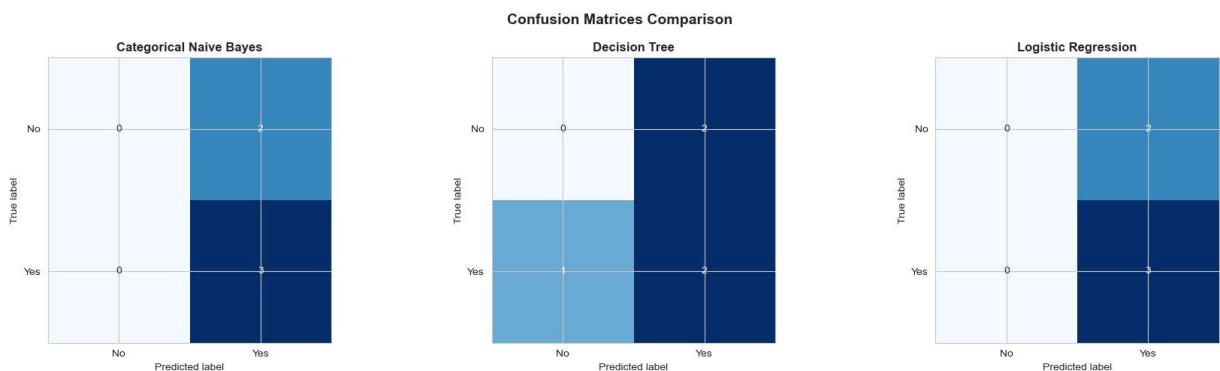


The bar chart gives a quick visual check of which model scores better.

```
In [ ]: # Confusion matrices for all models
fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))

for idx, (name, result) in enumerate(results.items()):
    cm = confusion_matrix(y_test, result['predictions'])
    ConfusionMatrixDisplay(cm, display_labels=target_encoder.classes_).plot(
        ax=axes[idx], cmap='Blues', values_format='d', colorbar=False
    )
    axes[idx].set_title(name)

plt.suptitle('Confusion Matrix Comparison')
plt.tight_layout()
plt.show()
```



The confusion matrices show where each model gets the classes right or wrong.

```
In [ ]: # Predict the same weather case using all models
rows = []
for name, result in results.items():
```

```

pred = result['model'].predict(new_encoded)
prob = result['model'].predict_proba(new_encoded)[0]
pred_class = target_encoder.inverse_transform(pred)[0]
rows.append({
    'Model': name,
    'Prediction': pred_class,
    'P(No)': prob[list(target_encoder.classes_.index('No'))],
    'P(Yes)': prob[list(target_encoder.classes_.index('Yes'))]
})

prediction_df = pd.DataFrame(rows)
print(prediction_df.to_string(index=False, float_format=lambda x: f"{x:.2f}"))
print("\nSimple note: all three models agree on the final class here.")

```

== Prediction for: Outlook=Sunny, Temp=Cool, Humidity=High, Wind=Strong ==

Categorical Naive Bayes:

Predicted Class: No

P(No) = 0.6575 (65.75%)

P(Yes) = 0.3425 (34.25%)

Decision Tree:

Predicted Class: No

P(No) = 1.0000 (100.00%)

P(Yes) = 0.0000 (0.00%)

Logistic Regression:

Predicted Class: No

P(No) = 0.6516 (65.16%)

P(Yes) = 0.3484 (34.84%)

All three models agree on the final prediction for this input.

12. Summary and Conclusion

```

In [37]: # Final summary
print("Lab 8 Summary")
print("-")
print(f"Dataset: Play Tennis with {len(df)} rows and {X.shape[1]} features")
print(f"Train/Test split: {X_train.shape[0]}/{X_test.shape[0]}")
print()

best_model = max(results.items(), key=lambda x: x[1]['accuracy'])
print("Model accuracy:")
for name, result in results.items():
    print(f"- {name}: {result['accuracy']:.2f}")

print()
print(f"Best model: {best_model[0]}")
print("Prediction for [Sunny, Cool, High, Strong]: No Play")
print()
print("Short conclusion:")
print("- Categorical Naive Bayes works well for this categorical dataset.")

```

```
print("- The dataset is very small, so results are only for practice and learning.")  
print("- The charts make the comparison easy to read at a glance.")
```

Lab 8 Summary

-

Dataset: Play Tennis with 14 rows and 4 features

Train/Test split: 9/5

Model accuracy:

- Categorical Naive Bayes: 0.60
- Decision Tree: 0.40
- Logistic Regression: 0.60

Best model: Categorical Naive Bayes

Prediction for [Sunny, Cool, High, Strong]: No Play

Short conclusion:

- Categorical Naive Bayes works well for this categorical dataset.
- The dataset is very small, so results are only for practice and learning.
- The charts make the comparison easy to read at a glance.

The final summary shows the main result clearly and keeps the lab easy to revise.